

FTS Docs

Project Title

Project Title:

Filial's Tracking Service (FTS)

Author(s):

Project Lead - Zach Schultz

Associates:

1. Michael Rauliuc
2. Ryan Bruno
3. Bilal Faruqi

Date Started: Jun 1, 2026

Course: ICS4UA.3

Last Updated: Jun 18, 2026

Date Delivered: Jun 18, 2026

Customer Requirements

Customer: David Moniaga

- He wants our ***SOULS***
- Son watches anime
- Wants a useful program
- Ooga Booga
- First meeting to discuss project requirements adjourned on Jun 1, 2026

Project Details

- Media tracker program where the user can track films, tv, and anime they are watching or have watched, as well as the date they have started or stopped watching it
 - Able to write reviews that are shared locally if they share watches
 - Able to view where to watch the show
- Program includes the following:
 - JFrame GUI for viewing and interaction

- API pulling from sources such as IMDB/TheMovieDB, TheTVDB to be stored into database
- Ability to fuzzy find media
- Ability to import and export watch-data onto a file
- Ability to have many user
- Ability to track media watched (Haven't Watch, Backlog, Completed, Drop)
- Ability to Score Media and write review
- Ability to See other user review

Rough Role

- Zach Schultz
 - Project Lead
 - Repo maintainer
 - UI Design(Making the Rough Images)
 - API key getter
 - Project Structure
 - DB Design
 - Quality Control
- Michael Rauliuc
 - Head of Documentation
 - UI Programmer Lead
- Ryan Bruno
 - Pull-Stack Engineer Co-Lead
 - API?
- Bilal Faruqi
 - Secrets Management
 - Pull-Stack Engineer Co-Lead
 - Lead Tester
 - Media Watcher

Project Timeline

	Requirement s	Design	Implementati on	Testing	Deployment
--	------------------	--------	--------------------	---------	------------

Proposed	Jun 1, 2026	Jun 2, 20... to Jun 3, 20...	Jun 3, 2026 to Jun 12, 2026	Jun 12, 20... to Jun 15, 20...	Jun 16, 20...
Completed	Jun 1, 2026	Jun 3, 20...	Jun 9, 2026	Jun 17, 20...	

Design Proposal

Filial's Tracking Service (FTS) is a local app that is able to track movies, shows, and anime that the users watch. Unlike normal tracking services, all the data is sorted locally using **SQLite**. All the Information such as Poster, Description, Rating will be pulled from the **TVDB/AniDb** (TVDB for movies/shows, and AniDb for anime). Then with the Use of **Jframe** as our front end, the User will be able to track which shows and movies they have watched and give their own personal rating. Then the user can keep using the program or start a new version of the FTS and **export** all of the information onto a new instance.

UI

Login Page

Login

Filial's Tracking Service

Username:

Password:

This will open by default and pressing either New User or Login will send you into the Homepage for the tracker

Create New User

Create new User

Create New User

Username:

Password:

Confirm Password:

maybe a check box if the Username is taken

When you press the **New User** Button it will send you to this page and it will let them create a new non admin user. After pressing Create user it will send back to the login page letting them now sign in

Home page

Homepage

Homepage Media List Search Settings X

Backlog Currently Watching Last Finished

Media List

Media List

HomepageMedia ListSearchSettingsX

Filter
Media Type

Status

Icon	Name	Status	Rating	Last EP	Rewatch
	Media 1				
	Media 2				
	Media 3				
	Media 4				
	...				

This page will display after pressing the media list button. It will display the media in a list with the ability to filter per type of media (movie, show, anime) and what status the media is in the tracker (backlog, currently watching, finished).

Media Page

Media List

The wireframe shows a media management interface. At the top is a navigation bar with buttons for 'Homepage', 'Media List' (which is highlighted), 'Search', 'Settings', and a close button 'X'. The main content area is divided into three columns. The left column contains a large box for a 'Poster' and two input fields for 'Start Date' and 'Finish Date'. The middle column displays the 'Title of Show' and 'show type', followed by a description: 'This show is about something with a guy with a REALLY big sword.' Below this are three input fields for 'Status', 'User Rating', and 'rewatches', and a 'Current EP' input field. The right column is a large box for 'User Reviews'. At the bottom, there are three buttons: 'Back', 'Save', and 'Add / Edit Review'.

Homepage	Media List	Search	Settings	X
Poster	Title of Show show type This show is about something with a guy with a REALLY big sword. Status User Rating rewatches Current EP	User Reviews		
Start Date				
Finish Date				
	Back	Save	Add / Edit Review	

This page will pop up any time a User Click on a Media, it will display extra information and also allow the user to change info about it such as if they finish it, their rating, total rewatches, and also see other users review of media or write their own

Review Pop Up?

Frame

Review for (---)

Cancel Save

This will be a pop up that lets the User write a quick review of the media

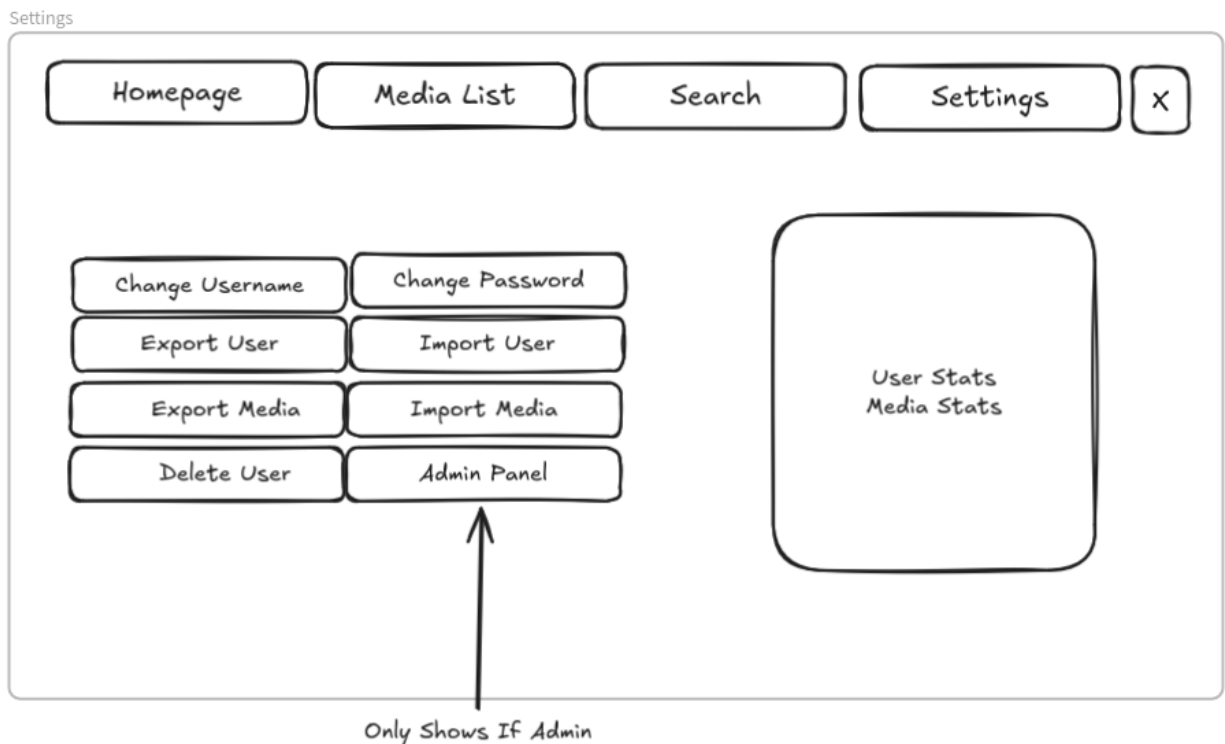
Search Page

Search

The sketch shows a web interface for a search page. At the top, there is a navigation bar with five buttons: 'Homepage', 'Media List', 'Search', 'Settings', and a close button 'X'. Below the navigation bar is a 'Search Box' followed by a 'Type Drop Down' menu and a 'Button'. The main content area contains a list of search results, each with a placeholder image box, a text field labeled 'Name', and a '+' button. A vertical line is on the left side of the list.

This section will show up when you click the search button. It will allow the user to enter the name of the show, what type the show is (movie, show, anime) then press the button to pull results from the db. From there the show will appear. Pressing the + button will add the media to the local DB and then it will change into a status picker (backlog, watching, completed)

Settings Page



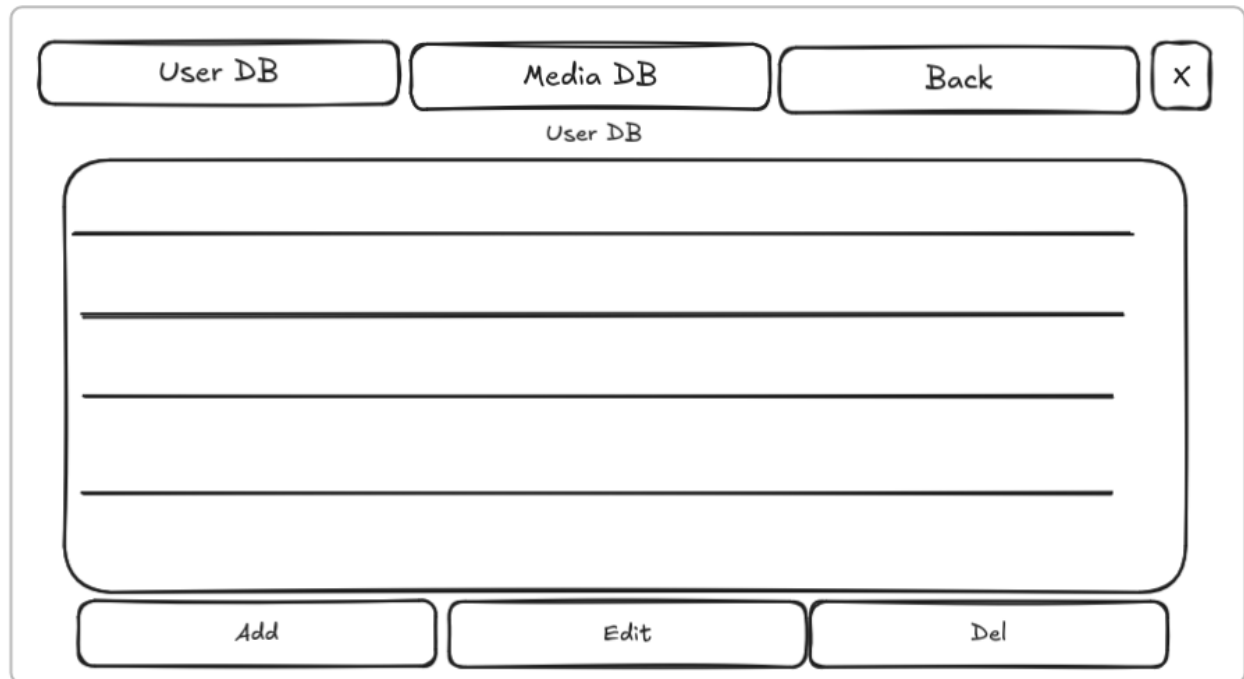
This Page will open up If a user Click the Settings Button, This is where the User can see stats about their account, such as total shows watched, or total item backlogged. There are also more settings to the left that lets them change their information like username and password. They can also export information from the database into a Json so they can import it onto another machine. Lastly there is a way for the user to delete themselves and a button that will open the Admin Panel **Only if you are admin**

Admin Panel

Note that the Header Changes in the Admin Panel and pressing the back button will take you back to the normal layout

User DB

User DB



A hand-drawn sketch of a 'User DB' management interface. At the top, there are four buttons: 'User DB', 'Media DB', 'Back', and a close button with an 'x'. Below these buttons, the text 'User DB' is centered. The main area is a large rounded rectangle containing four horizontal lines, representing a list of users. At the bottom, there are three buttons: 'Add', 'Edit', and 'Del'.

The User DB is the page that lets admin edit other users in the app, it will allow them to make new users like in the login page, give other people admin perms, and delete users that aren't using the software anymore.

Media DB

User DB

User DB

Media DB

Back

X

Media

Wipe DB

Re fetch Media Data

Add

Edit

Del

This page will let the admin change data about the Media, if something breaks this will let them go fix it. This also lets them completely remove everything off the db for storage saving or to start fresh.

API

When the user searches for media, our program will be extracting data from TheTVDB and AniDB, grabbing the name of the media, its description, its ID, release date, the poster image, and type of media it is (movie, show, anime).

TheTVDB

<https://thetvdb.github.io/v4-api/> (specific implementation)

- We will create an API key and pull:
 - Artwork
 - Episode count
 - Movie and series status
 - Content rating
 - ID of media

AniDB:

<https://wiki.anidb.net/API> (main doc page)

https://wiki.anidb.net/UDP_API_Definition (specific implementation)

We will implement via the UDP API version to pull data from anime. It requires sending requests from Datagram Packets in java, to send requests. It also requires a login for AniDB.

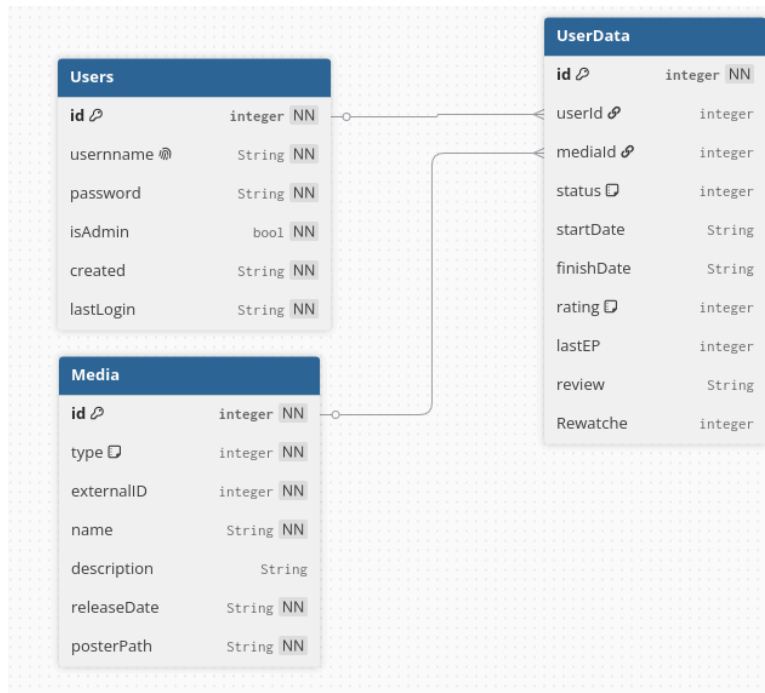
- AniDB can pull data via general release, tags, start days, other names, etc.
 - Description gets pulled separately(along with everything else below)
 - Specific calendar release days
 - Character/Creator information
 - Episode information
- Data is pulled via bit value assignment
(https://wiki.anidb.net/UDP_API_Definition#Data_Commands)
- Basic setup is provided as https://wiki.anidb.net/UDP_API_Definition#Basics

Limitations:

- General results state issues regarding poster pull from AniDB, and would be preferable to fall back to TheTVDB
- There must be slow request commands to ensure ip does not banned
- Requires an account to login to services (prevents bots) and will be provided by our team

DB

This is the Current Plans for the SQL tables



We have 3 Tables, Being Users, Media, and UserData

Users

This Table Holds all the Information about the User in the app. It will have a password, username, id etc.

Media

This Table Holds all the Information about the Media that is being sorted locally, it holds the name, description, location of image and what type of media it is. All of this will be pulled via the API

UserData (Temp Name)

This Table Holds information about the user related to the media they are watching. Info such as what they are watching, when they started watching, when they finished, what ep they ending on, etc. It will be linked to both the Media and Users to find the relative information

Importing & Exporting

Because the FTS is a local program, if you ever wanted to start again on another program the FTS has an Inbuilt way to import and export both user information and

media data. This will allow you to start another FTS setup and let it import all the media that was tracked or import a full user and all of their information.

Multi-User

The FTS Will have the ability to handle multiple users with there being 2 types. Normal Users that will have all the base features for the FTS (tracking, adding new media, review etc) and then Admin user which will have all the previous features and also some more admin tools such as clearing information off the database, editing other users and have full control over the application.

Implementation Details and Deliverables

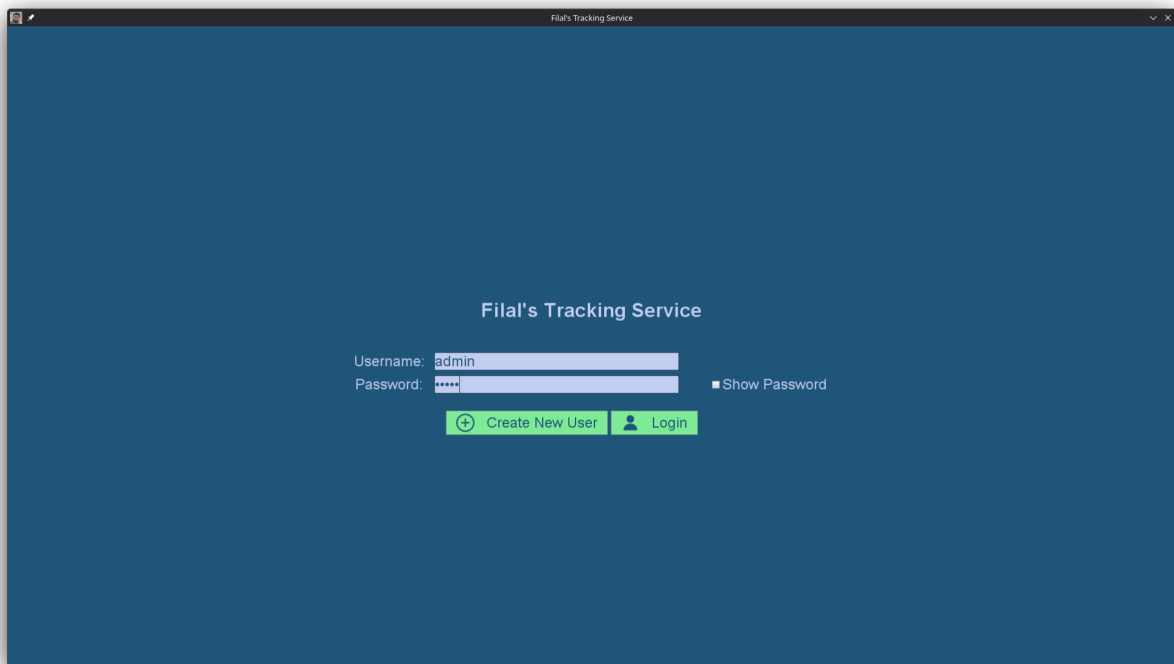
Watchthrough of Program

The best way to go over what this Program can do is take you on a Step by Step watchthrough of how to use the FTS.

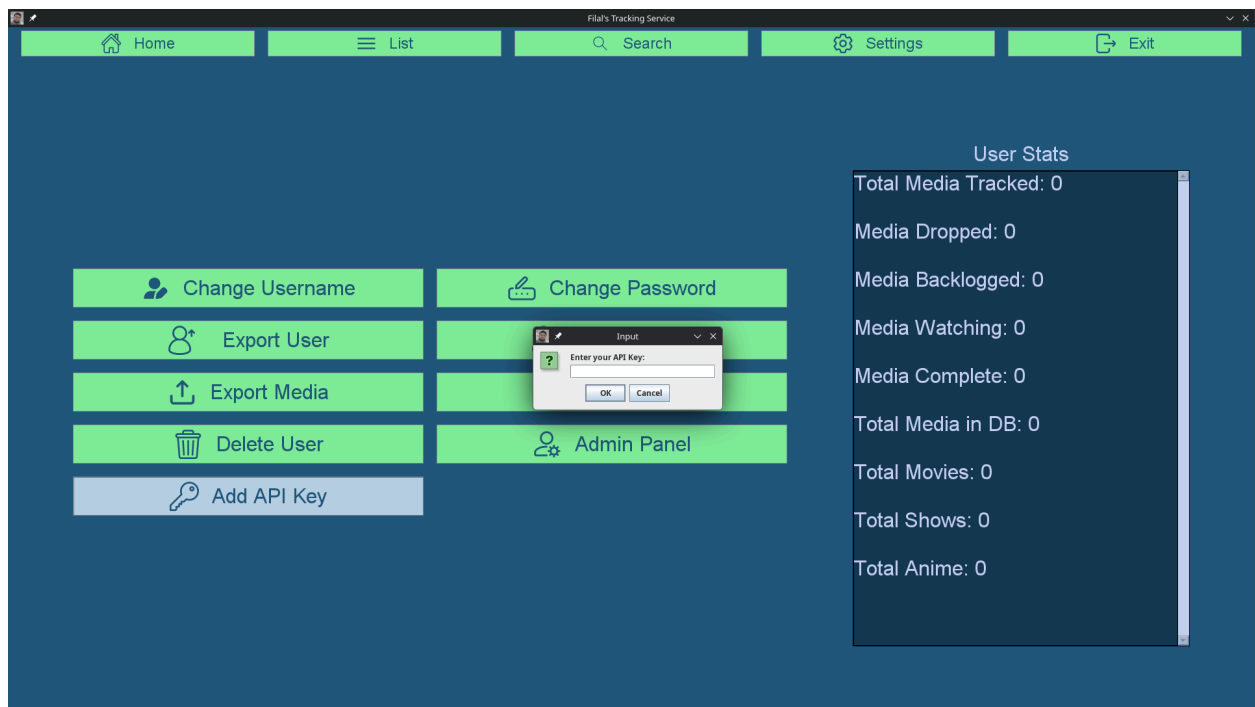
Setup

Before Using the FTS there are some setup tasks you need to do before using the Program Namely Adding your TheTVDB API Key into the program. This can be done with logging into the default admin account, going to Settings -> Add API Key

Login Page:



Settings Page:

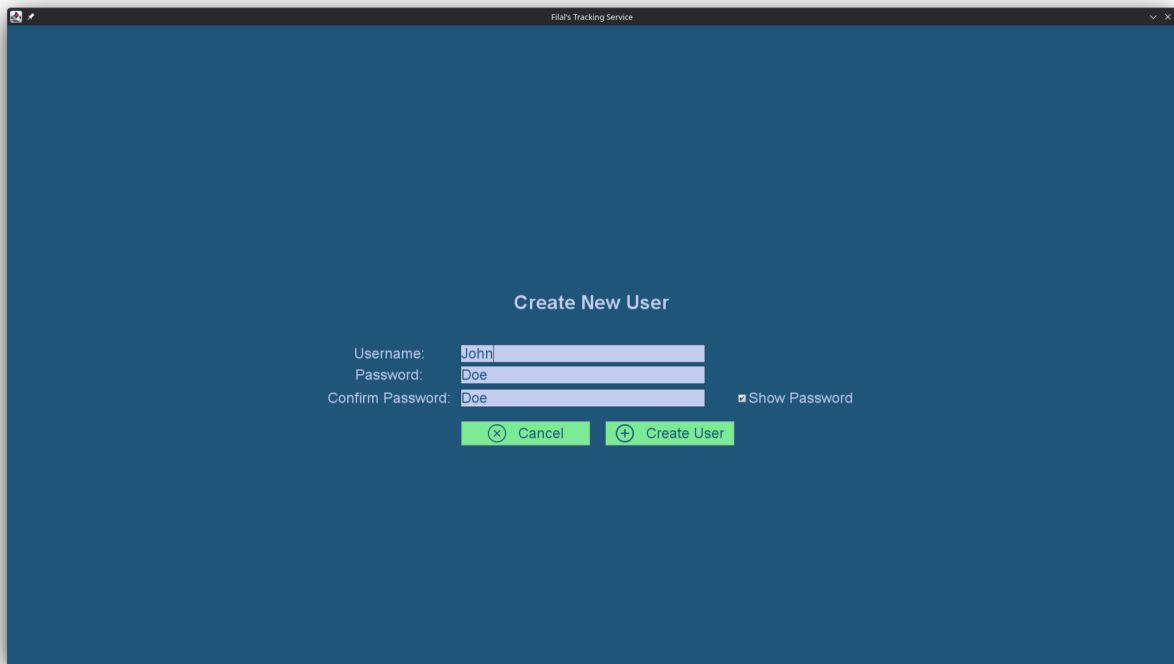


Media DB Page:

Enter API Key:

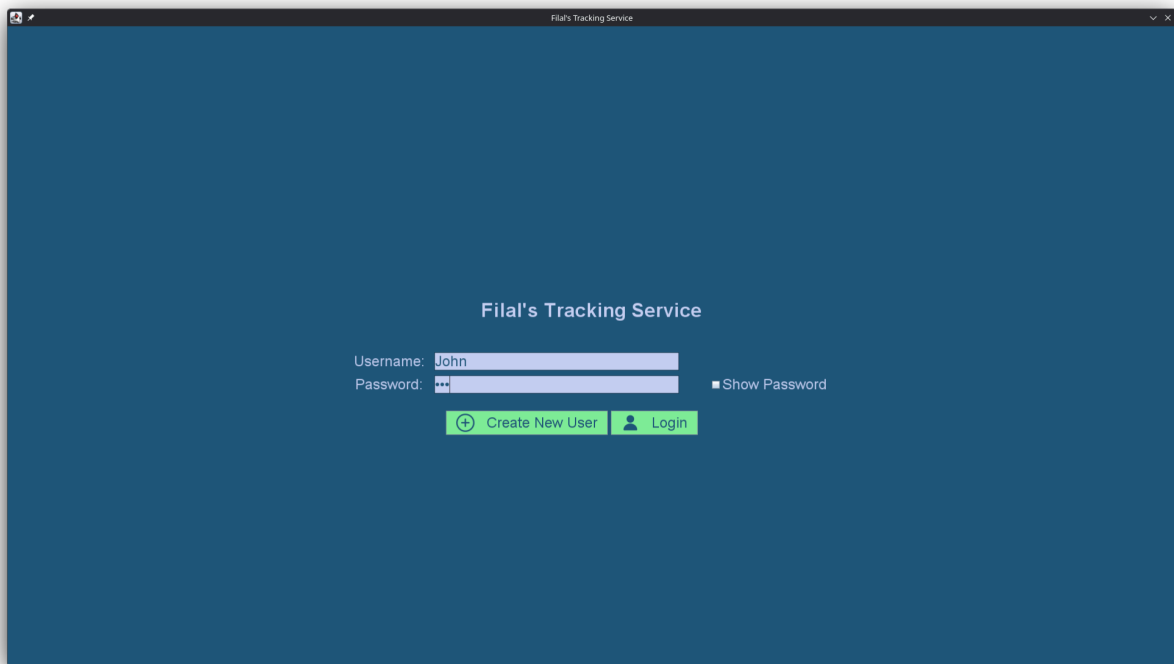
From here the API Key has been set you can press the logout button to exit admin and create a new account. This can be done by pressing Exit -> Create New User -> Enter Values into Fields, -> Create User

User Creation:



The screenshot shows a web application window titled "Flat's Tracking Service". The main content area has a dark blue background. In the center, there is a white "Create New User" form. The form contains three input fields: "Username:" with the value "John", "Password:" with the value "Doe", and "Confirm Password:" with the value "Doe". To the right of the "Confirm Password:" field is a checkbox labeled "Show Password". Below the input fields are two green buttons: "Cancel" (with a red 'x' icon) and "Create User" (with a green '+' icon).

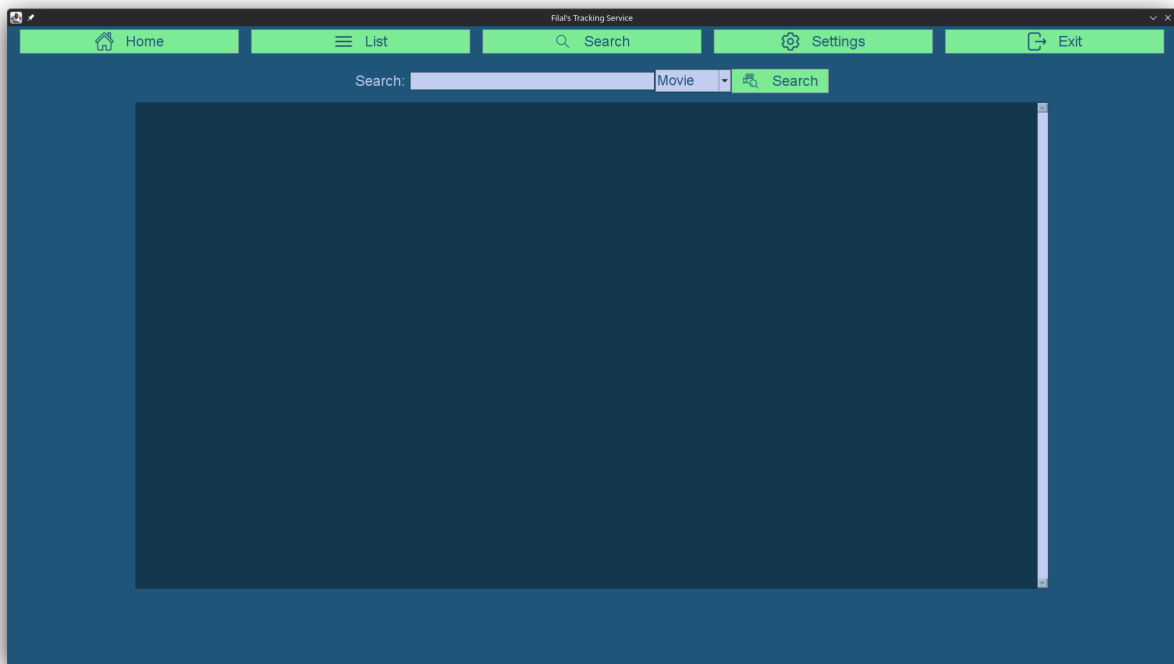
Login as New user:



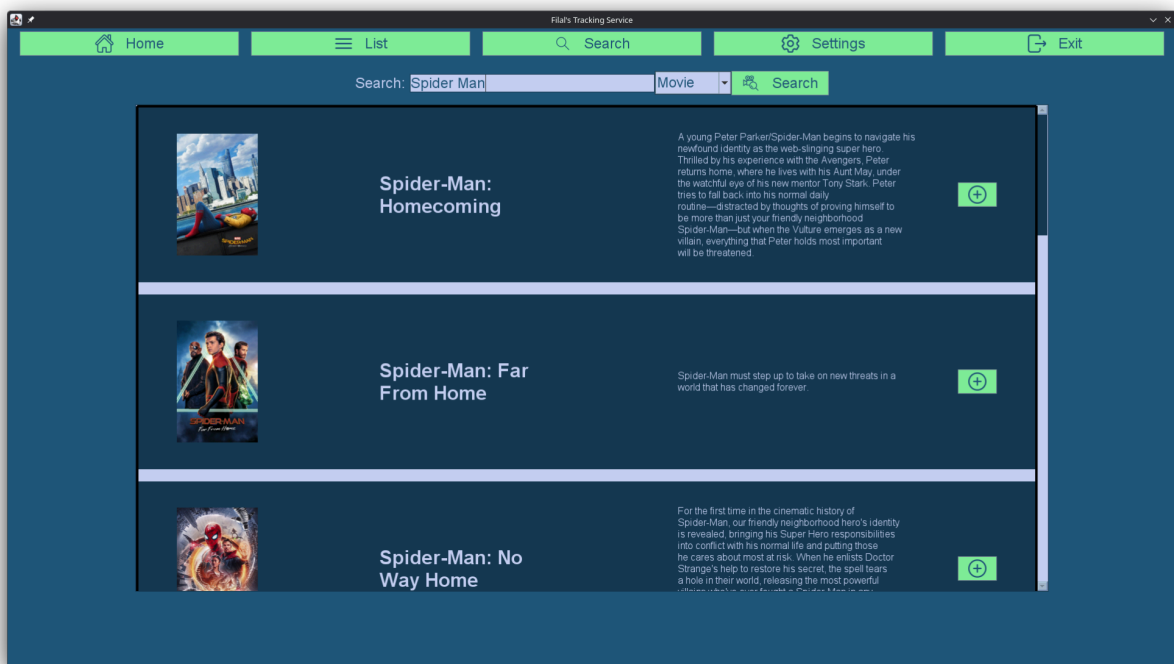
Adding Media

Now that we have added the API Key and created a new user, we can start adding our own media to the tracker. This can be done on the search page where you can search for the 3 different types of media we support. (Movies, TV Shows, Anime) By pressing the Search button or the enter Key will start a Search for the top 10 results based on your query. Once you find the media you are looking for, press the Big + button to the right to add the media onto the local DB. This will store all the information about the show and download the poster image directly onto your computer. Then for quick access you can adjust your status with the media, saying you are currently watching it, already finished it, or just leave it as undecided. Note that any media downloaded on Search will be added to all users however the status will only change what the current user status is.

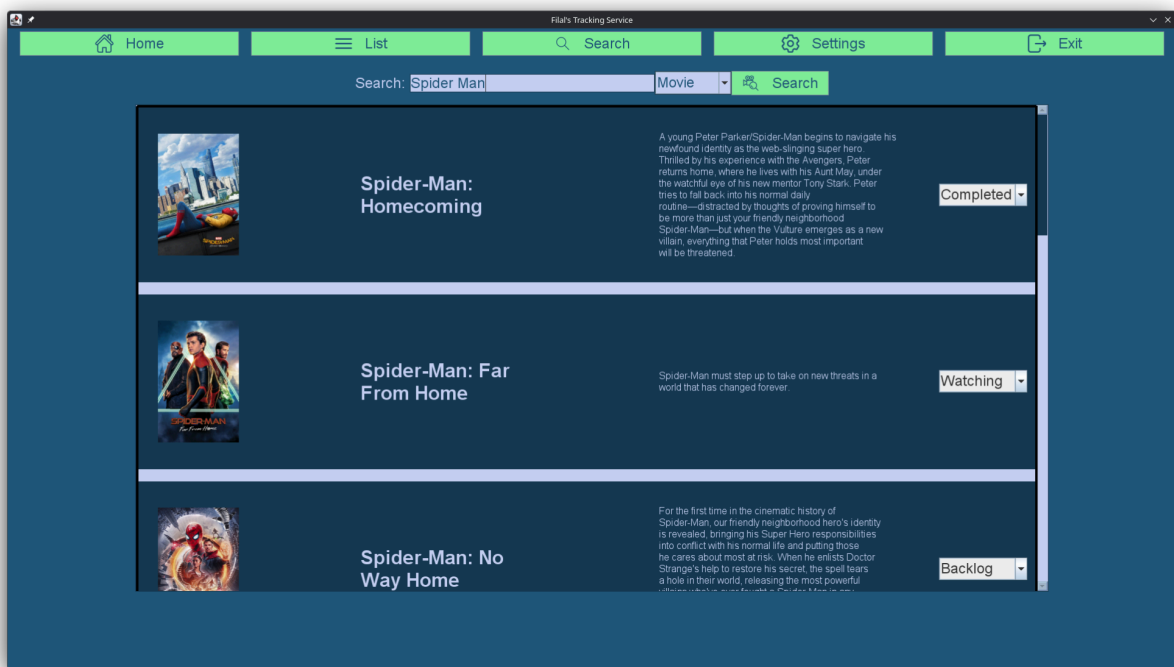
Search Page:



Results for “Spider Man”:



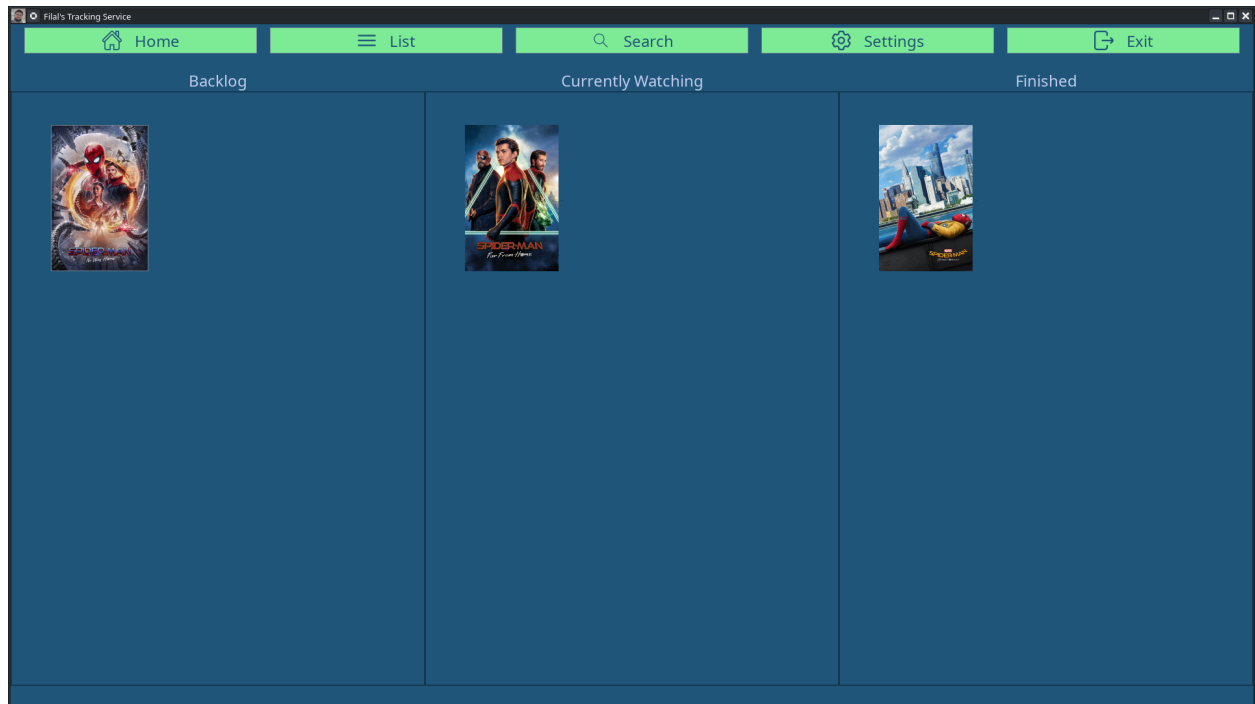
After Clicking + and Setting Status:



Viewing Media

Now that you have added media to the tracker, you now have 2 ways of viewing it. First is on the homepage as any show mark with Backlog, Watching, or Completed will be found with a quick click there. You can click and media to enter its Media page (which will be discussed later)

Home Page:



If you want to see the rest of the media or just see this in a more compact view, you can go onto the List Page where the FTS will give you the power to Filter and Search for any media in your DB base on Name, Type of Media (Movie, TV Show Anime), Status (Undecided, Dropped, Backlog, Watching, Completed) and Rating (0-10 and Note that 0 is Default Rating). Any Filter Changes you make can be Reset with the Reset Button. Once you find the media you want, press the Open media button to enter the Media Page.

Base List Page:

Home

List

Search

Settings

Exit

Movie:

TV Show:

Anime:

Name:

Status: All

Min Rating: 0

Max Rating: 10

Search

Reset

Open Media

Icon	Name	Status	Rating	Last EP	Rewatch
	Spider-Man: Homecoming	Completed	0	1	0
	Spider-Man: Far From Home	Watching	0	0	0
	Spider-Man: No Way Home	Backlog	0	0	0

Filtering For Media:

Home

List

Search

Settings

Exit

Movie:

TV Show:

Anime:

Name: Homecoming

Status: C

Min Rating: 0

Max Rating: 10

Search

Reset

Open Media

Icon	Name	Status	Rating	Last EP	Rewatch
	Spider-Man: Homecoming	Completed	0	1	0

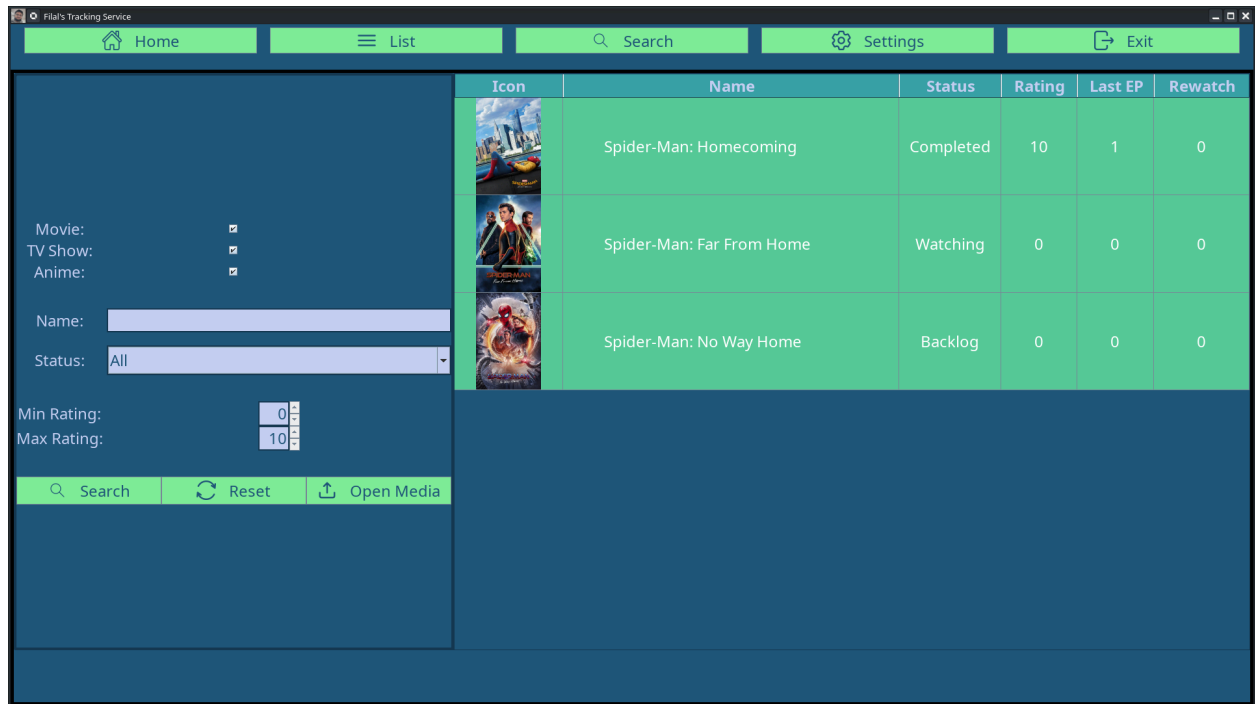
Editing Tracking

Now either from the homepage or List page you have entered the media page for a given piece of media. Here you can re-adjust your status and now edit your rating of a media, the total number of times you have rewatched the media and the last episode that you watch (note that for movies, they all have 1 EP by default). Note that when setting your status to watching, the program will automatically set the start date to today's date. Like wise, setting the show to Completed will set today's date as when you finish the media and also max out the episode count to the final episode. Any changes made will only save after pressing the save button on the bottom of the screen. Note that if you set your status back to undecided this will remove all watch data you have on a show and likewise you are unable to save your rating, episode count or review without first setting a Status other than undecided.

Media Page After Setting Rating to 10:

The screenshot shows the 'Spider-Man: Homecoming' media page in 'Fila's Tracking Service'. The interface has a dark blue theme with green navigation bars at the top and bottom. The top bar contains 'Home', 'List', 'Search', 'Settings', and 'Exit'. The bottom bar contains 'Back', 'Save', and 'Add/Edit Review'. The main content area is divided into three sections: a left sidebar, a central media details section, and a right 'User Reviews' section. The left sidebar shows a movie poster for 'Spider-Man: Homecoming', a 'Start Date' field with a placeholder 'yyyy-mm-dd', an 'End Date' field with the value '2026-06-18', and a 'Current Episode' field with the value '1'. The central section displays the movie title 'Spider-Man: Homecoming', the type 'Movie', a synopsis, and tracking fields for 'Status' (set to 'Completed'), 'Your Rating' (set to '10'), 'Rewatch' (set to '0'), and 'Current Episode' (set to '1'). The right section, titled 'User Reviews', displays 'No user reviews.'

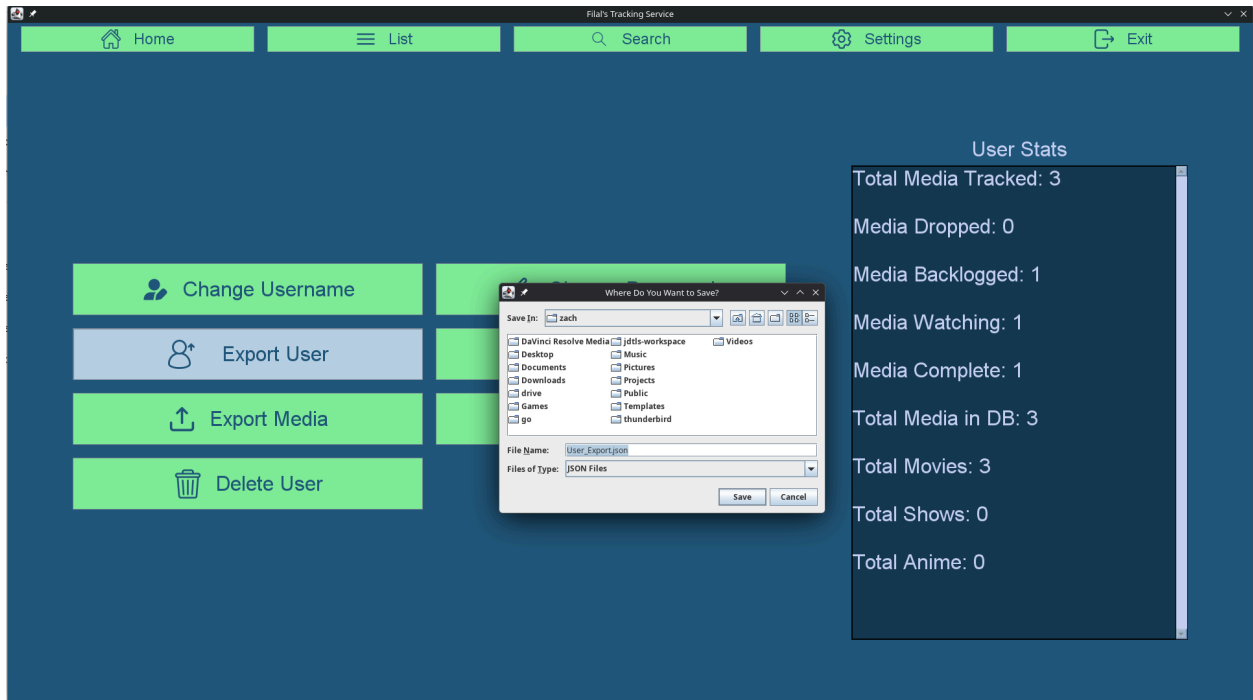
Change being updated on List:



Exporting

Now that we have created our User, we want to export this data either as a backup or to move to another setup. This can be done by going back onto settings and pressing the Export Button. The Media Button Export all Media that is sorted in the database, including stuff the user has not watched. The export user takes all the information about the user, (username, password, creation date) and also all the media they have watched and all status and reviews that come with them. This means if you export this user, when we import it, they will also come with all 3 spiderman movies.

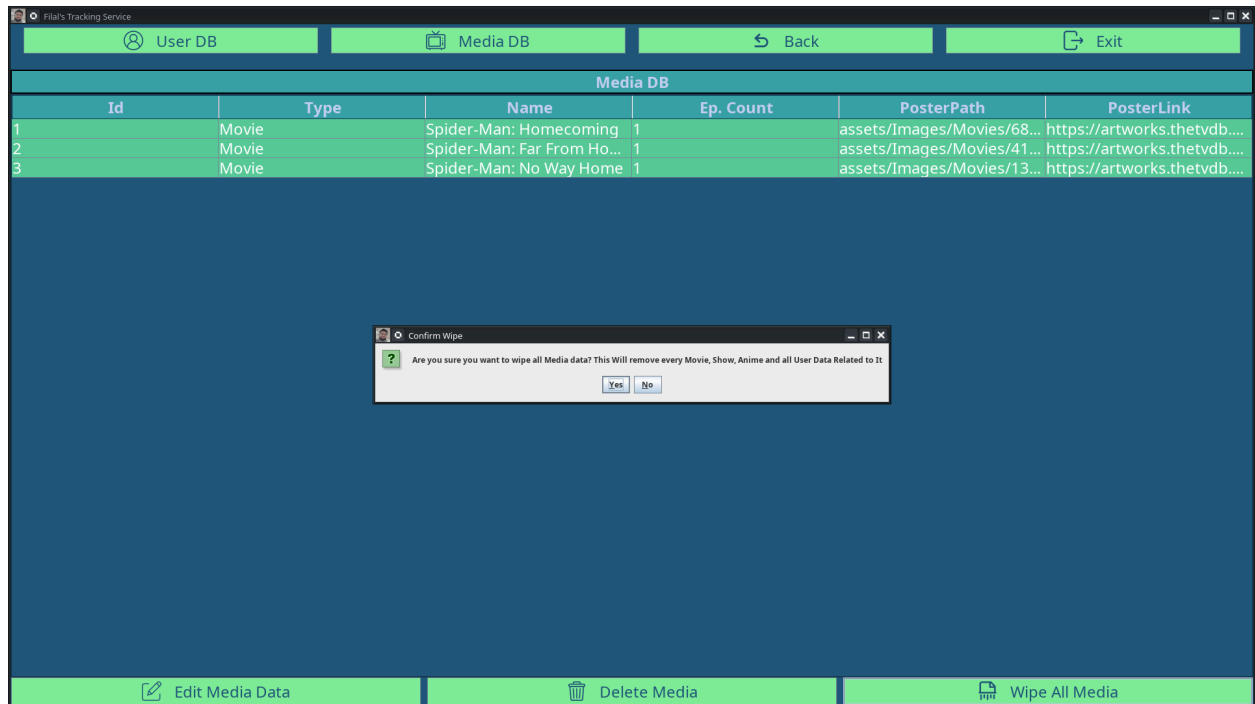
Exporting User:



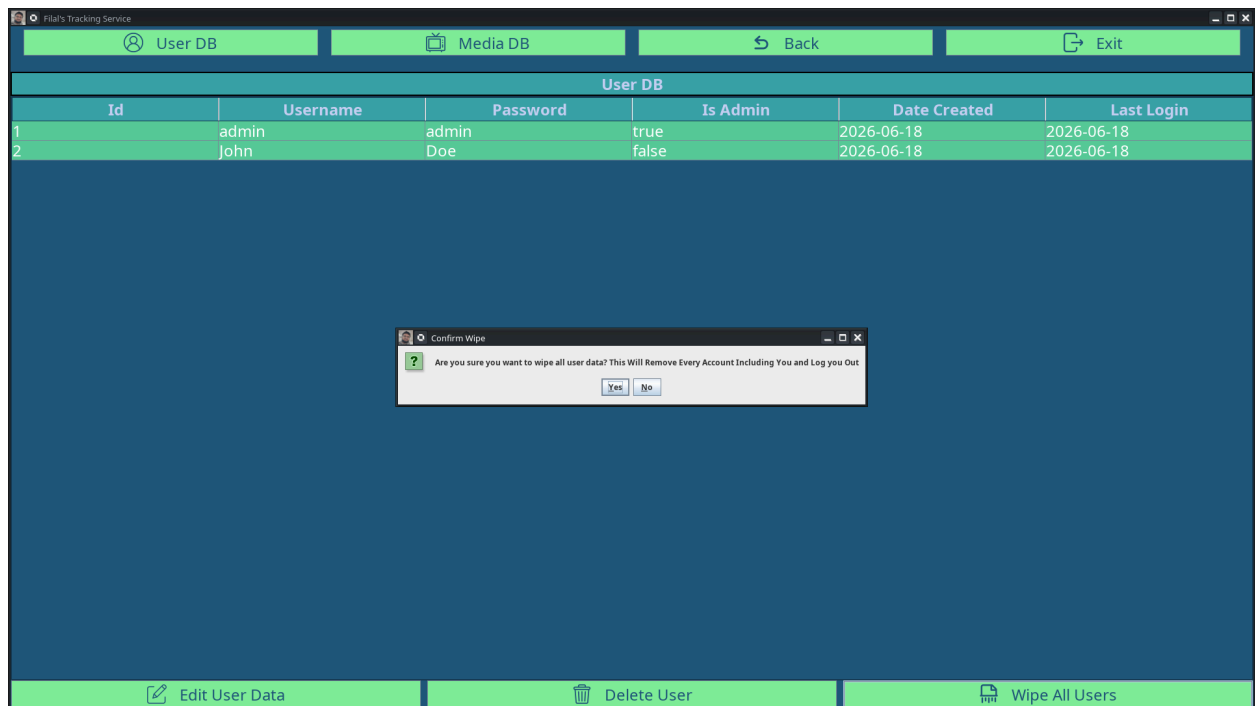
We will Test this Export User File Later so hold onto it.

Admin Control

Now if we logout and back onto our Admin account we can enter our Admin Panel again to see some new information. To enter the Admin Panel Log out of the current User and back onto Admin, go to settings and open the Admin Panel. From here we can remove all the Data on the DB then all of the Users
Admin Media Panel Wipe:



Admin User Pane Wipe:

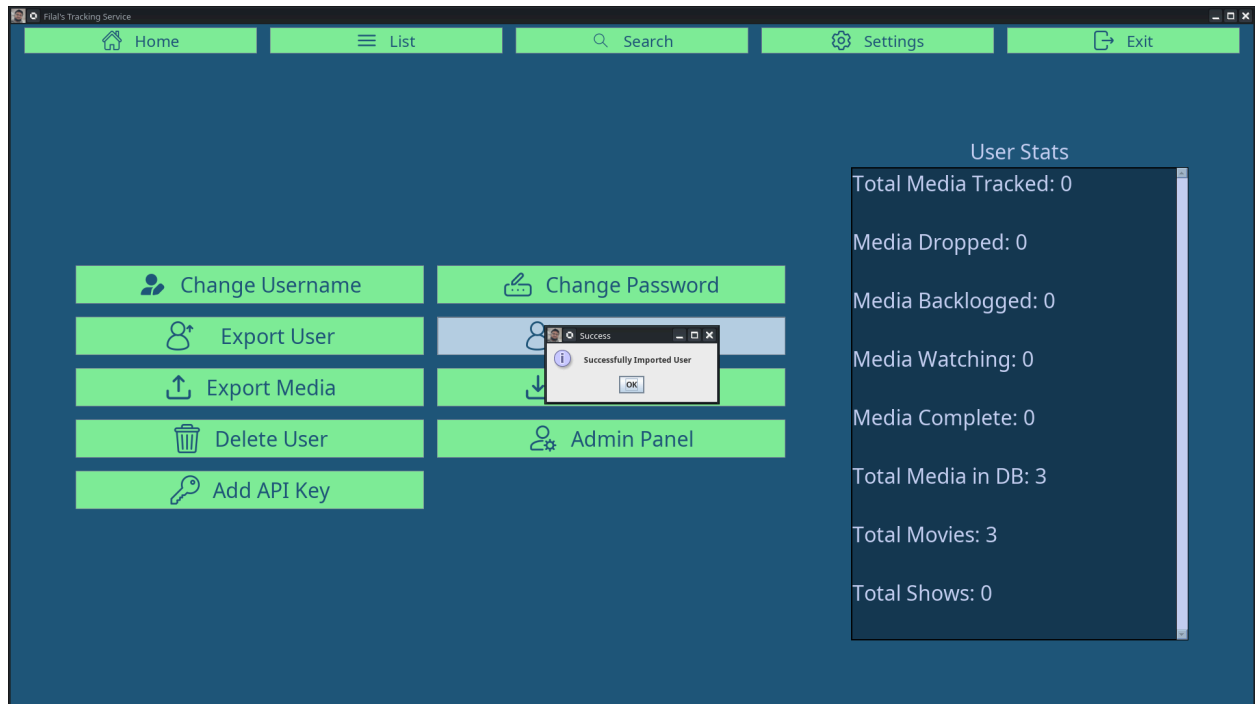


Importing ser

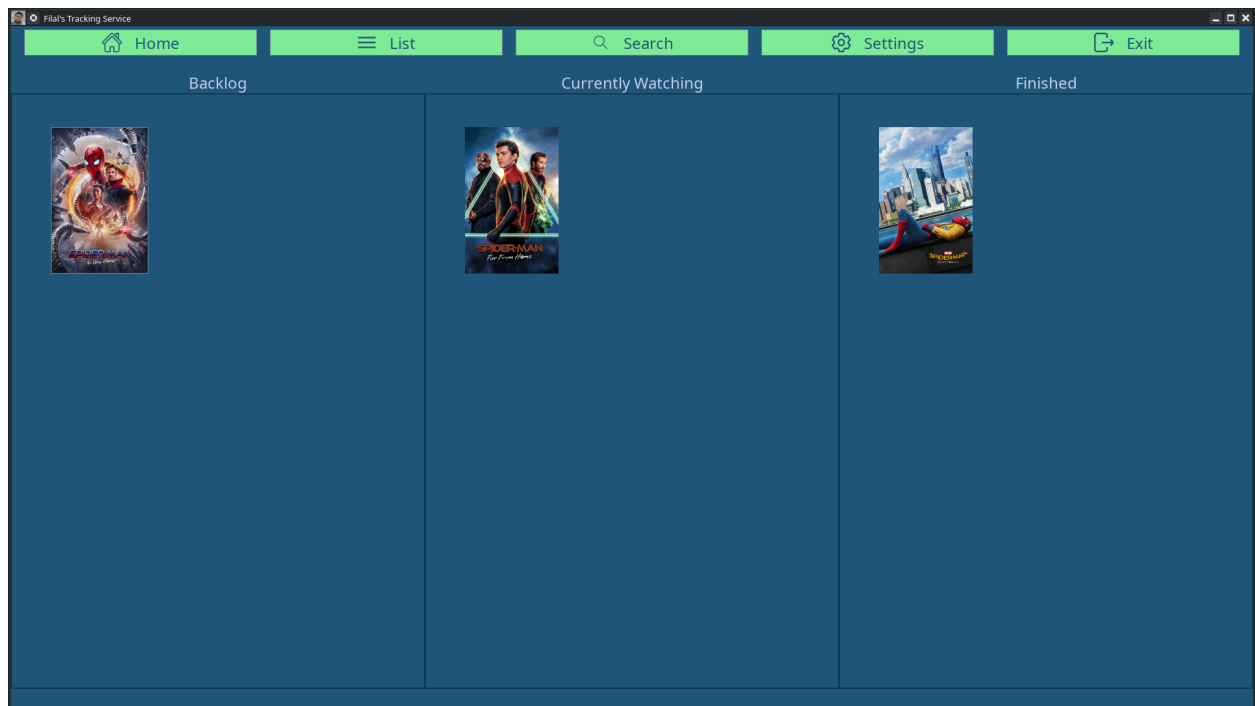
Now that the Full DB has been removed, we can relog back into admin and then

import the user

Importing User:



Relog as User:



As we can see all the information that was just removed was now readdd because of Importing.

Project Structure

Project Home

- |— **assets** (Where Image are Stored)
 - | |— **Images** (Poster Images)
 - | | |— **Anime** (Anime Poster)
 - | | |— **Movies** (Movie Poster)
 - | | |— **Shows** (Show poster)
 - | |— **UI** (Where the icons for the buttons are located)
- |— **db** (The main database storage location)
 - | |— **Tables** (Unused Backup for the Tables)
- |— **docs** (Extra docs & this project doc.)
- |— **key** (Where the API Key are found)
- |— **lib** (Where our Extra Library are found)
- |— **src** (Where the Java file are Found)
 - | |— **API** (API Usage)
 - | |— **DB** (Local DB Control)
 - | |— **DTO** (Data Transfer Object)
 - | | |— **API** (Files to Format the api Query)
 - | | | |— **Request** (Format for API Request)
 - | | | |— **Response** (Format for API Reponses)
 - | | |— **LocalDB** (Format for Local Object)
 - | |— **UI** (Files for creating JFrame and implementing app function)
 - | | |— **Pages** (Files for all the different pages)
 - | |— **Util** (The one lonely util class file that we needed)

Deployment and Maintenance

As with any project, there are still features and improvements that could be made with more time. A major feature we never got to, but would be fairly important, is encryption for a user API key. The program should be fine without this because it is local, meaning that any sensitive information such as the TheTVDB API is safe as long as no one can remote into the device. However, setting up some encryption so that no one can see the API key other than the admin account would be a nice security feature especially if you don't trust the other users on the device. Another nice feature would be having a search be combined between Movies, Anime, and TV shows which in theory can be done but with some issues. The main problem is that to do this we either need to only use TVDB which has worse metadata for anime or none at all, or try and mix the output of the 2 API which is 100% possible but you come into issue of the same show display twice or the relevance to your search being worse (for example it pulling a Anime that has a very similar name to a Movie you want). Some general things to examine back at, where given more time we would hope to improve and professionalize the formatting and layout for all the elements, as they work and are functional but could use extra touchups. Other than that, as mentioned above, it would be best if there was a way to work with the api key better and encrypt. One thing we are proud of is the eventual look and feel of the homepage especially, because it looked rough for a while, during the process of setting up the rest of the program functionality, but eventually led to a nice feel that works and integrates everything together. An example of this was our homepage, where for a very long while was rough and unpolished but as we worked on our project and improved we were able to come back to it and make it more polished and functional. Additionally, the usage of git allowed us to work better and more efficiently as a team, as we were able to assign tasks to each other and work on them remotely, until coming together to merge our branches. Some of our core strengths as a team was our communication and organization, there were never any major conflicts and we were all able to understand our tasks at hand. We were able to discuss problems and support each other when needed. We were also able to

professionally discuss requirements and client needs without confusion and all understood what had to be done, how to do it, and when it had to be done.

Resource Allocations

Task	Responsibility	Description
Project Manager	Zach Schultz	Overlook the Project and both Physically with members and digital with being responsible with the remote code base.
Project Structure	Zach Schultz	Creating the file tree for the project and keeping the project organized
UI Design	Ryan Bruno & Zach Schultz	The Creation of the Original design sketches used to create the UI
DB Design	Zach Schultz	Creating the local tables and designing how we store data
UI Structure	Zach Schultz	Creating the layout for JFrame that every pages uses
UI Main Creation	Michael Rauliuc , Ryan Bruno , Bilal Faruqi	Writing the Java to Display the UI on every page (Home, Media, List etc). While this is one task, this took more time than almost every other task. Pages: Login Michael Rauliuc Create User Michael Rauliuc Main Page: Michael Rauliuc & Ryan Bruno List Page Michael Rauliuc

		Search Page Michael Rauliuc Setting Page Ryan Bruno Media Page Michael Rauliuc & Bilal Faruqi Admin Panels Michael Rauliuc
UI Clean Up	Ryan Bruno	Work on Theming and Icons for Each Page
UI Function	Zach Schultz	Added Functionality to UI Button and other Function that Link Front end to Backend
Database Connection	Zach Schultz	Working with Java to talk back and forth with the UI. Query needed information for the Project
API	Zach Schultz & Bilal Faruqi	Connection to External Server for Information on Media
Image Downloader	Zach Schultz	Download Images from Links
Importing & Exporting	Zach Schultz	Ability to Export User Information or Media Info you already download and Import it onto another Device
M.O.M. Writer	Michael Rauliuc & Ryan Bruno	Took notes during meeting to keep track on what we discuss
Project Doc Writer	Zach Schultz & Michael Rauliuc	Wrote up Everything you see on this Project Doc
Branding	Filal Baruqi	For the Use of his Branding for the Project

		and Likeness for Missing Images
--	--	---------------------------------

Reference(s)

External Library

- The JDBC Connection to Sqlite was done thanks to this project
 - <https://github.com/xerial/sqlite-jdbc?tab=Apache-2.0-1-ov-file>
- GSON: A Tool that lets us turn Json into Object and Object into Json
 - <https://github.com/google/gson>

Icons & Images

- Plus symbol: `<a href="https://www.flaticon.com/free-icons/plus-sign" title="plus sign icons">Plus sign icons created by HAJICON - Flaticon</a>`
- Exit icon: `<a href="https://www.flaticon.com/free-icons/logout" title="logout icons">Logout icons created by Freepik - Flaticon</a>`
- Login icon: `<a href="https://www.flaticon.com/free-icons/person" title="person icons">Person icons created by Febrian Hidayat - Flaticon</a>`
- X icon: `<a href="https://www.flaticon.com/free-icons/quit" title="quit icons">Quit icons created by Pixel perfect - Flaticon</a>`
- Home icon: `<a href="https://www.flaticon.com/free-icons/home-page" title="home page icons">Home page icons created by Aldo Cervantes - Flaticon</a>`
- List icon: `<a href="https://www.flaticon.com/free-icons/list" title="list icons">List icons created by Ilham Fitrotul Hayat - Flaticon</a>`
- Search icon: `<a href="https://www.flaticon.com/free-icons/search-bar" title="search bar icons">Search bar icons created by Tanah Basah - Flaticon</a>`
- Setting icon: `<a href="https://www.flaticon.com/free-icons/settings" title="settings icons">Settings icons created by Freepik - Flaticon</a>`
- Bin icon: `<a href="https://www.flaticon.com/free-icons/trash" title="trash icons">Trash icons created by Freepik - Flaticon</a>`

- Export icon: Export icons created by Dixit Lakhani_02 - Flaticon
- Import icon: Import icons created by Dixit Lakhani_02 - Flaticon
- Change icon: Refresh icons created by Freepik - Flaticon
- Admin icon: Setting icons created by Tanah Basah - Flaticon
- TV search icon: Tv icons created by Iconsmeet - Flaticon
- Movie search icon: Magnifier icons created by Boris farias - Flaticon
- Anime search icon: Girl icons created by Freepik - Flaticon
- Change username icon: Change name icons created by Kreev Studio - Flaticon
- Change password icon: Otp icons created by HideMaru - Flaticon
- Export user icon: Export icons created by Lagot Design - Flaticon
- Import user icon: Import icons created by Lagot Design - Flaticon
- User Db icon: User icons created by Freepik - Flaticon
- Media db icon: Tu tv icons created by Freepik - Flaticon
- Back icon: Back arrow icons created by Syahrul Hidayatullah - Flaticon

- Save icon: `<a href="https://www.flaticon.com/free-icons/save" title="save icons">Save icons created by Yogi Aprellyanto - Flaticon</a>`
- Review icon: `<a href="https://www.flaticon.com/free-icons/chat-box" title="chat box icons">Chat box icons created by Graphics Plaza - Flaticon</a>`
- Edit icon: `<a href="https://www.flaticon.com/free-icons/edit" title="edit icons">Edit icons created by Kiranshastry - Flaticon</a>`
- 🔥 icon: `<a href="https://www.flaticon.com/free-icons/ok" title="ok icons">Ok icons created by bsd - Flaticon</a>`
- Wipe db icons: `<a href="https://www.flaticon.com/free-icons/shred" title="shred icons">Shred icons created by Maniprasanth - Flaticon</a>`
- Api key icon: `<a href="https://www.flaticon.com/free-icons/password" title="password icons">Password icons created by Freepik - Flaticon</a>`

External Help

- Any and all AI tools that were used to research, help understand, or use as autocomplete for this project.
- No Code (To the Team's Knowledge) was fully vibed coded however it would be a lie to say this project would have been completed without these tools